REPORTPILOT

Evidence-grounded research and report-generation workspace

Developed By

Vedang R. Dhuri

(BE. Information Technology)

COMPLETE SOFTWARE PROJECT REPORT

Abstract

ReportPilot is a web application for organizing research questions and producing reviewable, source-grounded reports. The inspected implementation combines a React and TypeScript browser interface with a Python FastAPI application, PostgreSQL persistence, a separate task worker, and a LangGraph workflow. A user creates a project and research brief, may upload project documents, queues a durable research task, and follows persisted stage events. The workflow plans the question, searches the web through Tavily, retrieves relevant project passages using PostgreSQL English full-text search, analyzes findings, assesses claim support, and writes a structured draft. A person reviews the report, can approve or reject it, can revise a previously reviewed version, and can export approved reports in Markdown, DOCX, or PDF. The project also implements owner-scoped workspace analytics, versioned report records, authentication and session controls, migrations, and mocked-provider tests. The five labels "planner," "researcher," "analyst," "verifier," and "writer" refer to graph stages, not five autonomous software agents. The researcher stage invokes a deterministic search adapter; the other model-assisted stages request structured outputs from the configured chat model. Retrieved text and model assessments are not guarantees of truth, and human review is an explicit part of the design. Testing evidence available for this report is 56 passing backend tests and a successful frontend type-check/build. The running Docker Compose API and frontend responded to health/HTTP checks. These results do not prove successful live research through external model and search providers; a previous planner attempt recorded a provider deadline failure.

Keywords: evidence-grounded research, report workflow, large language model, human review, LangGraph, full-text search, PostgreSQL, source provenance, asynchronous processing.

Chapter 1 — Introduction

1.1 Background

Research work typically includes framing a question, locating information, selecting relevant evidence, comparing claims, drafting a synthesis, and communicating limitations. A conventional manual process distributes these activities across browsers, files, notes, and word processors. An unreviewed language-model answer can be faster to obtain, but a reader may find it difficult to inspect which source passages support a claim or to reproduce the steps behind a polished summary. ReportPilot addresses a practical workflow problem rather than claiming to solve research as a whole. It gives each signed-in user an owner-scoped project space; saves briefs, uploaded documents, tasks, event history, source passages, claim assessments, report versions, and review decisions; and keeps generated work subject to human review. The system combines a web search provider and an optional set of project documents with model-generated structured planning and synthesis.

1.2 Application domain and intended users

The application belongs to the domain of research workflow support and source-aware document generation. Its interface is suited to an individual researcher, student, analyst, or team member who wants to organize questions and inspect a report draft. The inspected implementation is owner-scoped; no team sharing, organization roles, or collaborative editing was found. The report therefore describes single-user ownership, not a multi-tenant team product.

1.3 Motivation and contribution

The implementation brings several stages into one persistent process: a question is captured as a structured brief; work runs outside the HTTP request lifecycle; source passages are stored and linked to claims; stage progress is replayable; report versions and decisions are recorded; and exports are gated on approval. The contribution is the integration of these engineering mechanisms into one application. It is not a new retrieval algorithm, language model, verification metric, or empirical research result.

1.4 Scope

In scope are account registration and login, projects and briefs, document upload and text extraction, PostgreSQL full-text retrieval, Tavily web search, configurable OpenAI or Gemini model calls, task queue and worker, structured research stages, evidence records, claim assessments, report review/revision/export, live events, analytics, local Docker Compose deployment, and the repository's backend tests

Chapter 2 — Problem Statement and Objectives

2.1 Problem statement

When research tasks are handled manually or by unstructured generative chat, evidence can become detached from conclusions, long-running work can be lost when held inside one request or browser view, and a reader may have no explicit mechanism to inspect and review a version before sharing it. ReportPilot implements a project-scoped workflow that persists research state and evidence, associates report content with known source records, and makes human review an explicit status transition. This problem statement describes the gap addressed by the source code. It does not claim that every conventional tool lacks citations, that ReportPilot is more accurate than another product, or that automated verification eliminates hallucinations.

2.2 Primary objective

Build a web-based system that organizes research inputs and produces a persistent, inspectable report draft from retrieved evidence, with progress tracking, human review, revision history, and approved-report exports.

2.3 Specific objectives

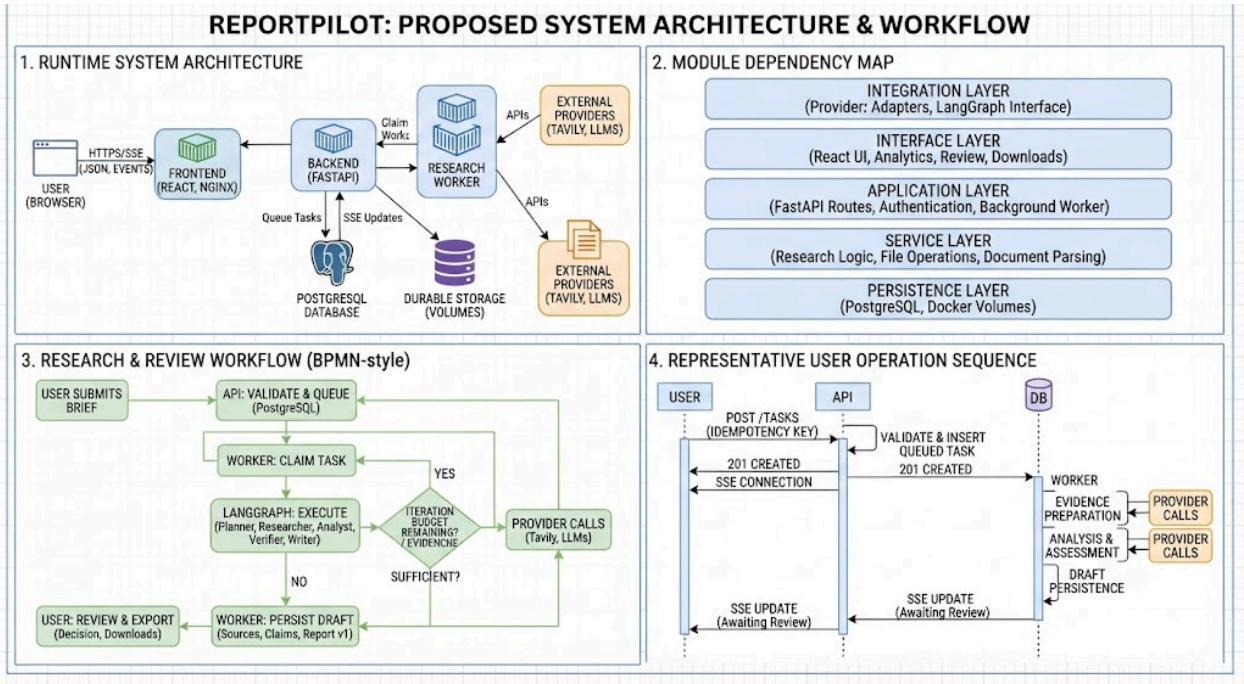
1. Organize projects, briefs, documents, and tasks under an authenticated owner.
2. Queue bounded research tasks and persist state changes and progress events.
3. Use configured search and chat-model providers to collect and synthesize evidence.
4. Persist source records and evidence passages and link findings to valid evidence identifiers.
5. Provide claim-level and revision-level AI assessments, visibly framed as fallible aids.
6. Require a human decision before export and preserve report revisions and decisions.
7. Support a browser UI for task monitoring, source inspection, review, and analytics.
8. Provide migrations, a local container baseline, and tests for key backend behaviors

2.4 Conventional approaches

A manual research workflow usually combines a browser, local documents, note-taking, a spreadsheet or citation manager, and a separate report editor. An ad hoc AI workflow can generate a draft quickly but may not persist the retrieval process or bind every statement to a retained passage. These are general process observations, not results from a controlled user study.

2.5 Proposed ReportPilot workflow

ReportPilot adds a durable project model and a tracked task lifecycle. It stores a brief before enqueueing work; the worker claims it from PostgreSQL; the graph gathers and analyzes passages; source, evidence, claim, and report records are persisted; and stage events are streamed to the browser. The report is initially pending human review, versions are retained, and an export endpoint checks approval state



1. Runtime System Architecture

- **Client & API Layer:** The browser runs a React frontend built with Vite and served via Nginx, sending authenticated JSON requests and consuming real-time updates through Server-Sent Events (SSE). FastAPI handles input validation, authentication dependencies, ownership checks, and task lifecycle management.
- **Asynchronous Workers & Queues:** FastAPI and the background research worker share a PostgreSQL database. The worker claims queued tasks using row-level locking (FOR UPDATE SKIP LOCKED) and advisory locks to maintain single-worker loop safety.
- **External Providers & State:** External model and search calls (such as Tavily and configured LLMs) are handled through dedicated provider adapters. LangGraph orchestrates the structured node execution while checkpointing state directly into PostgreSQL.

2. Module Dependency Map

The application is structured into distinct, decoupled software layers:

- **Integration Layer:** Manages provider adapters and the LangGraph execution interface.
- **Interface Layer:** Encapsulates the React UI components, analytics, report review dashboards, and export download utilities.
- **Application Layer:** Contains FastAPI route handlers, authentication dependencies, and the background worker service.

- **Service Layer:** Encapsulates core business logic including research workflows, file operations, and document parsing/chunking.
- **Persistence Layer:** Manages PostgreSQL database models, transactions, and Docker volume persistence.

3. Research & Review Workflow (BPMN-Style)

- **Ingestion & Queueing:** The user submits a research brief, which FastAPI validates and persists as a queued task in PostgreSQL.
- **Bounded Iteration Graph:** The worker claims the task, recovers its graph state from database checkpoints, and triggers the research graph. The graph executes a bounded iterative cycle through planning, source retrieval via Tavily, analysis, and claim verification.
- **Verification & Persistence:** The verifier filters out unsupported claims lacking valid evidence. Once complete, the worker persists the sources, passages, claims, verifications, and Report Version 1 into the database, transitioning the task to an awaiting-review status streamed via SSE to the user.

4. Representative User Operation Sequence

- **Idempotent Submission:** The user submits a request to the task endpoint with an idempotency key. The API validates project ownership and inserts a queued task record, returning a 201 Created response along with an active SSE connection.
- **Background Execution:** The research worker claims the task from the queue, executes evidence preparation and analysis steps via external provider calls, and processes intermediate states asynchronously without keeping the HTTP connection blocked.
- **Live Notifications:** Upon completing draft persistence, an SSE update notifies the browser that the report is ready for human review and export.

2.6 Technology Stack and Tools

Technology / observed version	Purpose and placement	Data received and produced / interaction
Python 3.11 container; local tests ran in Python 3.14	Backend language; `backend/app/`.	Receives HTTP/database/provider data; produces API responses, tasks, and report records. The version difference matters when reproducing test runs.
FastAPI 0.162.0	API routing, dependencies, validation, OpenAPI; `backend/app/api/`.	Typed request/response data and DB dependencies; returns JSON, SSE, or downloads.
Uvicorn 0.54.0	ASGI application server; backend Docker command.	Runs `app.main:app` on port 8000.
Pydantic 2.14.1; pydantic-settings 2.16.1	Validates request/output schemas and environment configuration.	Converts input/model outputs to constrained Python models; rejects invalid values.
SQLAlchemy 2.1.6; psycopg 3.3.6	ORM and PostgreSQL driver; `backend/app/models/`, `database.py`.	Maps records and compiles SQL; PostgreSQL returns rows and commits.
PostgreSQL 16 image	Durable application DB and full-text search; Compose.	Stores owners, projects, briefs, tasks, evidence, reports, events, document chunks, sessions, and checkpoints.
Alembic 1.20.1	Versioned schema migrations; `backend/alembic/`.	Migration scripts change database schema.
LangGraph 0.6.11; `langgraph-checkpoint-postgres` 2.0.25	State graph, routing, checkpoint support; `agents/graph.py`, `workers/research.py`.	Typed task state; returns node updates and can resume graph checkpoint by task thread ID.
`langchain-openai` 0.3.35; `langchain-google-genai` 2.1.12	Provider wrappers for selected chat model; `providers/llm.py`.	Prompts and structured schema constraints; provider returns a model result. External execution is not proven by mocked tests.
Tavily Search API	Web retrieval; `providers/search.py`.	Query and result limit; returns validated result title, URL, passage, and date fields.
HTTPX 0.28.1	Synchronous HTTP provider requests; `providers/search.py`.	Sends Tavily request with configured timeout; maps network/status failures to provider errors.

Technology / observed version	Purpose and placement	Data received and produced / interaction
React 18.3.1, React DOM 18.3.1, TypeScript 5.7.3	Browser UI and type checking; `frontend/src/`.	UI state/API values become interactive pages; compile-time type errors fail the build.
Vite 6.4.5; React Vite plugin 4.7.0	Development/build pipeline; `frontend/vite.config.ts`.	Builds TypeScript/React assets for Nginx.
React Router DOM 7.18.4	Browser history/navigation wrapper; App reads current location and navigates to screen paths.	Path/query selects overview, research, library, analytics, or task screen.
TanStack Query 5.105.0	Server-state cache, polling, invalidation; `main.tsx` and pages.	API queries/mutations, refresh intervals and invalidation events.
React Hook Form 7.91.0; Zod 4.6.7; resolver 5.9.2	Form state and client validation.	Form input becomes typed submit values or visible validation errors. Server validation remains authoritative.
React Markdown 10.1.0; Lucide React 0.468.0	Render report Markdown/citations and UI icons.	Report Markdown and application state become UI elements.
Tailwind CSS 3.4.19; PostCSS 8.5.32; Autoprefixer 10.6.2	Styling toolchain; application stylesheet uses custom CSS and Tailwind imports.	CSS sources transform to browser styles.
pypdf 5.9.0; python-docx 1.2.0	Extract PDF/DOCX text and generate DOCX output.	Uploaded document bytes to text; report records to DOCX bytes.
ReportLab 4.5.1	Generate application PDF exports and this academic report.	Structured report/text paragraphs to searchable PDF. Application export uses Letter; this companion report uses A4.
PyJWT 2.15.1; pwdlib 0.3.1 with Argon2 recommended hasher	Access-token signing/validation and password hashing.	User ID/expiry to signed token; password to salted hash and verification result.
pytest 8.4.2	Backend tests.	Test inputs/stubs to assertions and pass/fail report.
Docker Compose; PostgreSQL 16, Python 3.11-slim, Node 22-alpine, Nginx 1.27-alpine	Local multi-container runtime.	Builds/starts backend, worker, database, and static frontend. No cloud deployment platform is configured.

Chapter 3- AI Agents and Automation

Stage	Responsibility and input	Output / decision	Model or tools	Implementation and failure behavior
Planner	Receives question, objectives, scope, source preferences, recency.	Bounded subquestions, up to 8 search queries, scope boundaries, outline.	Configured chat model with `ResearchPlan` output schema; no direct tools.	Graph node; exceptions propagate to task failure and stage failure event.
Researcher	Receives plan and optional supplementary queries plus local document sources.	Deduplicated source list and incremented iteration.	Deterministic Tavily adapter; max 8 queries per pass and configured result cap.	Python function, not an LLM agent. Provider errors fail workflow.
Analyst	Receives research question, plan, source passages.	Up to 40 claim drafts, limitations, open questions, supplementary queries.	Configured chat model with `AnalysisOutput`.	Unknown source IDs are filtered.
Verifier	Receives findings and source evidence.	Per-claim status and explanation.	Configured chat model with `VerificationOutput`.	Claims missing evidence are forcibly unsupported; missing assessments become not_checked.
Writer	Receives question, language, outline, findings, assessments, evidence, limitations.	Structured report summary, sections, methodology, limitations, source IDs.	Configured chat model with `ResearchReport`.	Unknown citation IDs are removed.

ReportPilot combines one configurable chat model, deterministic provider/service functions, and a fixed state graph. The code names graph nodes planner, researcher, analyst, verifier, and writer. They are not five independently instantiated agents with separate memory, tool permissions, planning loops, or inter-agent messages. The graph invokes a model factory for structured model stages. The researcher uses a Python search function and merges results. The graph has sequential edges and one conditional loop back to research. Parallel agent execution and model-controlled routing are not implemented. An AI agent normally implies a system that observes context, chooses or invokes actions, and can continue toward a goal with some autonomy. An LLM call is one model inference. A tool is an operation exposed to an agent or invoked by application code; here Tavily is called by deterministic researcher code. A workflow is an application-defined set of nodes and transitions. An ordinary function deterministically transforms inputs subject to its code. These distinctions avoid describing the graph as a multi-agent system without evidence

Chapter 4— Installation, Configuration, and Execution

4.1 Docker Compose

Apply migrations before starting the application, as documented in the repository README:
docker compose run --rm backend alembic upgrade head
docker compose up --build
Open http://localhost:5173
API docs http://localhost:8000/docs
liveness http://localhost:8000/health/live
Readiness http://localhost:8000/health/ready

4.2 Local development

python -m venv backend\.venv
backend\.venv\Scripts\Activate.ps1
python -m pip install -r backend\requirements.txt
python -m alembic -c backend\alembic.ini upgrade head
python -m uvicorn app.main:app --app-dir backend --reload

In a second terminal:
backend ..\.venv\Scripts\Activate.ps1
python -m app.workers.research

In a third terminal:
npm --prefix frontend install
npm --prefix frontend run dev

Conclusion -

ReportPilot implements a persistent, owner-scoped research workflow that connects a browser UI, FastAPI routes, PostgreSQL records, a background worker, a LangGraph state graph, Tavily search, configurable LLM providers, document processing, report review, and approved exports. Its key engineering characteristics are durable task state, persisted progress events, bounded graph iteration, linked evidence records, model-assisted claim assessments, versioned reports, human review, and an interactive analytics dashboard. The repository's backend tests and frontend build passed during report preparation, and the Compose API and frontend passed basic runtime health/HTTP checks. These facts establish exercised local behavior, not live model success or research accuracy. A prior planner request failed with a provider deadline, and the final provider path was not independently demonstrated here. Human source inspection remains essential. ReportPilot is therefore best described as an inspectable research-assistance workflow—not an autonomous researcher or a factuality guarantee.

References -

[1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems (NeurIPS), 2020. [Online]. Available: https://arxiv.org/abs/2005.11401. DOI: 10.48550/arXiv.2005.11401.

[2] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” International Conference on Learning Representations (ICLR), 2023. [Online]. Available: https://arxiv.org/abs/2210.03629. DOI: 10.48550/arXiv.2210.03629.

[3] FastAPI, “FastAPI Documentation.” [Online]. Available: https://fastapi.tiangolo.com/. Accessed: 29 Oct. 2026.

[4] OWASP Foundation, “Prompt Injection.” OWASP Community. [Online]. Available: https://owasp.org/www-community/attacks/PromptInjection. Accessed: 29 Oct. 2026.

[5] PostgreSQL Global Development Group, “8. Full Text Search,” PostgreSQL Documentation. [Online]. Available: https://www.postgresql.org/docs/current/textsearch.html. Accessed: 29 Oct. 2026.

[6] LangChain, “LangGraph Documentation: Overview.” [Online]. Available: https://docs.langchain.com/oss/python/langgraph/overview. Accessed: 29 Oct. 2026.

[7] Pydantic, “Pydantic Documentation.” [Online]. Available: https://docs.pydantic.dev/latest/. Accessed: 29 Oct. 2026.

[8] SQLAlchemy, “SQLAlchemy 2.0 Documentation.” [Online]. Available: https://docs.sqlalchemy.org/en/20/. Accessed: 29 Oct. 2026.

[9] React, “React Documentation.” [Online]. Available: https://react.dev/learn. Accessed: 29 Oct. 2026.

[10] TanStack, “TanStack Query Documentation.” [Online]. Available: https://tanstack.com/query/latest/docs/framework/react/overview. Accessed: 29 Oct. 2026.

[11] Tavily, “Search API Reference.” [Online]. Available: https://docs.tavily.com/documentation/api-reference/endpoint/search. Accessed: 29 Oct. 2026.